KEVIN C. JONES - 4N6POST.COM | SQL

SQL Cheat Sheet

A quick-reference guide to Structured Query Language, split into two levels: **Beginner** fundamentals on the left (selecting, filtering, grouping, joining) and **Advanced** queries on the right (subqueries, window functions, CTEs, and performance). Examples use generic SQL syntax that works on MySQL, PostgreSQL, MSSQL, and SQLite with minor dialect tweaks.

How to Use This Sheet

0

Logical order of operations: `FROM` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `ORDER BY` → `LIMIT`.

Beginner — SELECT & FROM

1

CLAUSE	PURPOSE	EXAMPLE
`SELECT`	Choose columns (or expressions) to return	`SELECT name, email FROM users;`
`SELECT *`	Return all columns (avoid in production)	`SELECT * FROM orders;`
`AS`	Alias a column or table	`SELECT salary * 12 AS yearly FROM emp;`
`DISTINCT`	Remove duplicate rows	`SELECT DISTINCT city FROM users;`
`FROM`	Source table(s); may be a subquery or join	`SELECT id FROM orders o;`

`SELECT 1+1;` works with no table (e.g., `SELECT NOW();`).

Beginner — WHERE & Operators

2

OPERATOR	MEANING	EXAMPLE
`= <> < > <= >=`	Comparison (use `<>` for "not equal")	`WHERE age >= 18`
`AND / OR / NOT`	Boolean logic; `AND` binds first	`WHERE city='X' AND age>21`
`IN (...)`	Match a list of values	`WHERE state IN ('CA','TX')`
`BETWEEN a AND b`	Inclusive range	`WHERE price BETWEEN 10 AND 50`
`LIKE`	Pattern match; `%` any, `_` one char	`WHERE name LIKE 'J%'`
`IS NULL / IS NOT NULL`	NULL check (never use `= NULL`)	`WHERE deleted_at IS NULL`

Beginner — ORDER BY, LIMIT, OFFSET

3

CLAUSE	PURPOSE	EXAMPLE
`ORDER BY col`	Sort ascending (`ASC`) by default	`ORDER BY name ASC`
`ORDER BY col DESC`	Sort descending	`ORDER BY created DESC`
`ORDER BY a, b`	Sort by multiple keys (left-to-right)	`ORDER BY last, first`
`LIMIT n`	Return at most `n` rows	`LIMIT 10`
`OFFSET n`	Skip `n` rows (pagination)	`LIMIT 10 OFFSET 20`

Postgres/MySQL: `LIMIT 10 OFFSET 20`. MSSQL uses `OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY` (requires `ORDER BY`).

Beginner — Aggregates & GROUP BY

4

FUNCTION	RETURNS	EXAMPLE
`COUNT(*)`	Number of rows	`SELECT COUNT(*) FROM users`
`COUNT(col)`	Count of non-NULL values	`COUNT(email)`
`SUM / AVG`	Total / average (numeric)	`SUM(amount), AVG(score)`
`MIN / MAX`	Smallest / largest	`MIN(price), MAX(price)`
`GROUP BY col`	Group rows per distinct value	`GROUP BY dept_id`
`HAVING`	Filter after grouping (aggregates)	`HAVING COUNT(*) > 5`

Every non-aggregated column in `SELECT` must appear in `GROUP BY`. Use `HAVING` for aggregate filters; `WHERE` runs before grouping.

Beginner — JOINs

5

JOIN	ROWS RETURNED	EXAMPLE
`INNER JOIN`	Only matching rows in both tables	`FROM a INNER JOIN b ON a.id=b.a_id`
`LEFT JOIN`	All rows from left + matches from right (NULL if none)	`FROM a LEFT JOIN b ON a.id=b.a_id`
`RIGHT JOIN`	All rows from right + matches from left	`FROM a RIGHT JOIN b ON a.id=b.a_id`
`FULL OUTER JOIN`	All rows from both sides (MySQL lacks this)	`FROM a FULL OUTER JOIN b ON a.id=b.a_id`
`CROSS JOIN`	Cartesian product of both tables	`FROM a CROSS JOIN b`

Alias tables (`FROM users u`) to keep joins readable. Always qualify columns with the table alias when a name is ambiguous (`u.id`, `o.id`).

Advanced — Subqueries

6

TYPE	USE	EXAMPLE
`Scalar subquery`	Returns one value used as a column or in `WHERE`	`SELECT (SELECT MAX(price) FROM p) AS top`
`IN / NOT IN`	Filter against a sub-result set	`WHERE id IN (SELECT user_id FROM orders)`
`EXISTS`	True if the subquery returns any row	`WHERE EXISTS (SELECT 1 FROM b WHERE b.a_id=a.id)`
`Correlated`	References outer query; runs per outer row	`WHERE price > (SELECT AVG(price) FROM p x WHERE x.cat=p.cat)`
`FROM subquery`	Derived table / inline view (must alias)	`FROM (SELECT … ) AS t`

Correlated subqueries are often slow — prefer a `JOIN` or a `LEFT JOIN ... IS NULL` for "not in" patterns.

Advanced — Window Functions

7

FUNCTION	PURPOSE	EXAMPLE
`ROW_NUMBER()`	Sequential number per partition	`ROW_NUMBER() OVER (ORDER BY created)`
`RANK() / DENSE_RANK()`	Ranking (gaps vs no gaps for ties)	`RANK() OVER (PARTITION BY dept ORDER BY sal DESC)`
`LAG(col) / LEAD(col)`	Previous / next row value	`LAG(price) OVER (ORDER BY date)`
`SUM() OVER (...)`	Running total (aggregate over window)	`SUM(amount) OVER (ORDER BY date)`
`PARTITION BY`	Reset the window per group	`OVER (PARTITION BY user_id ORDER BY date)`

Window functions run after `WHERE`/`GROUP BY` but do not collapse rows. Fetch the latest row per group with `ROW_NUMBER() OVER (PARTITION BY g ORDER BY t DESC)` wrapped in a subquery.

Advanced — CTEs (WITH)

8

FEATURE	PURPOSE	EXAMPLE
`WITH name AS (...)`	Named subquery, reusable in the main query	`WITH recent AS (SELECT …) SELECT * FROM recent`
`Multiple CTEs`	Chain several named queries with commas	`WITH a AS (...), b AS (...) SELECT …`
`Recursive CTE`	Walk hierarchical data (org charts, trees)	`WITH RECURSIVE tree AS (…)`
`Readability`	Replace deeply nested subqueries	`WITH x AS (...) SELECT * FROM x`

CTEs improve readability and can be referenced multiple times. A recursive CTE needs an anchor `SELECT` + `UNION ALL` + recursive term with a termination condition.

Advanced — Indexes & Performance

9

TOPIC	GUIDELINE	EXAMPLE
`CREATE INDEX`	Speed up lookups on filter/join columns	`CREATE INDEX idx_users_email ON users(email);`
`Composite index`	Leftmost prefix rule applies	`CREATE INDEX ON t (a, b, c);`
`EXPLAIN`	View the query plan	`EXPLAIN ANALYZE SELECT …;`
`sargable`	Avoid wrapping columns in functions	`WHERE date >= …` not `WHERE YEAR(date)=…`
`SELECT only needed`	Fewer columns → less I/O	`SELECT id, name` not `SELECT *`

Non-sargable patterns (wrapping indexed columns in functions, leading `%LIKE%`) defeat indexes. Use `EXPLAIN` to confirm the plan uses them.

Advanced — Transactions & DML

10

STATEMENT	PURPOSE	EXAMPLE
`BEGIN / COMMIT`	Wrap writes so they apply atomically	`BEGIN; … COMMIT;`
`ROLLBACK`	Undo the transaction on error	`ROLLBACK;`
`INSERT`	Add rows	`INSERT INTO users (name) VALUES ('A');`
`UPDATE`	Modify existing rows	`UPDATE users SET active=1 WHERE id=3;`
`DELETE`	Remove rows	`DELETE FROM users WHERE id=3;`

Always scope `UPDATE`/`DELETE` with a `WHERE` clause — omitting it affects every row. `INSERT ... ON CONFLICT` (Postgres/SQLite) and `INSERT ... ON DUPLICATE KEY UPDATE` (MySQL) handle upserts.